

SAGA test specifikacija

Uvod i forma testova

Pet faza SAGA toka:

#	Forward	Kompenzator
F1	Rezervacija $qty \times strike$ na računu kupca	C1: oslobađanje rezervacije
F2	Provera i rezervacija qty hartija kod prodavca	C2: oslobađanje rezervacije
F3	Transfer sredstava od kupca do prodavca	C3: povratak sredstava na račun kupca
F4	Prenos vlasništva nad hartijama od prodavca do kupca	C4: vraćanje hartija prodavcu
F5	Ažuriranje stanja, ugovor prestaje da bude "važeći"	C5: vraćanje statusa na "važeći"

Promene stanja kroz faze i njihove kompenzacije:

- **F1:** kupac.available -= $qty \times strike$, kupac.reserved += $qty \times strike$. Sredstva ostaju na računu kupca, samo izolovana iz potrošivog dela.
- **C1:** inverzija F1: kupac.available += $qty \times strike$, kupac.reserved -= $qty \times strike$.
- **F2:** prodavac.holding[simbol].available -= qty , prodavac.holding[simbol].reserved += qty . Hartije i dalje kod prodavca, samo izolovane.
- **C2:** inverzija F2.
- **F3:** kupac.reserved -= $qty \times strike$, prodavac.available += $qty \times strike$. Sredstva od kupca stižu prodavcu. Upisuje se red u bank.transactions.
- **C3:** prodavac.available -= $qty \times strike$, kupac.available += $qty \times strike$. Suprotan transfer; novi red u bank.transactions (sa oznakom kompenzacije).
- **F4:** prodavac.holding[simbol].reserved -= qty , kupac.holding[simbol].quantity += qty . Vlasništvo nad hartijama se menja.
- **C4:** kupac.holding[simbol].quantity -= qty , prodavac.holding[simbol].reserved += qty . Suprotan prenos.
- **F5:** ugovor.status više nije "važeći".
- **C5:** vraćanje statusa na "važeći".

Kompenzatori se pozivaju obrnutim redom u odnosu na forward sekvencu. Moraju biti idempotentni i ponavljaju se sve do uspeha.

Orkestrator održava SAGA log: po jedan red po toku, sa poljima *saga_id*, *status* (*Running*, *Completed*, *Compensating*, *Compensated*), *current_step* (redni broj poslednje pokušane faze) i *log* (niz zapisa, jedan po pokušaju faze, sa forward i kompenzacionim koracima i njihovim ishodima).

Tranzicije:

- *Running* postaje *Completed* na uspešnom F5
- *Running* postaje *Compensating* na bilo kojoj grešci posle pisanja loga
- *Compensating* postaje *Compensated* kad svi potrebni C_i uspeju

Pre kreiranja loga orkestrator proverava postojanje ugovora, autorizaciju pozivaoca, status ugovora i *settlement date*. Ako pukne, vraća HTTP grešku; ne pravi se log i nema bočnih efekata.

Provera resursa (sredstva u F1, hartije u F2) je posle kreiranja loga; F1 ili F2 neuspeh ostavlja log sa statusom *Compensated* i *current_step* jednakim rednom broju neuspele faze.

Zapis u log

Primer za "happy path":

Trenutak	Status	current_step	Log
Posle F1	Running	1	[{F1: ok}]
Posle F2	Running	2	[{F1: ok}, {F2: ok}]
Posle F3	Running	3	[{F1: ok}, {F2: ok}, {F3: ok}]
Posle F4	Running	4	[{F1: ok}, ..., {F4: ok}]
Posle F5	Completed	5	[{F1: ok}, ..., {F5: ok}]

Primer ako F3 pukne i kompenzacija prolazi:

Trenutak	Status	current_step	Log
Posle F1	Running	1	[{F1: ok}]
Posle F2	Running	2	[{F1: ok}, {F2: ok}]
F3 pukao	Compensating	3	[{F1: ok}, {F2: ok}, {F3: err}]
C2 prošao	Compensating	3	[..., {F3: err}, {C2: ok}]
C1 prošao	Compensated	3	[..., {C2: ok}, {C1: ok}]

Forma svakog testa

Setup. TRUNCATE i reseed.

Action. Jedan HTTP POST na /options/\$ID/exercise, potencijalno sa adversarnim headerima.

Wait. Poll loga do terminalnog statusa (*Completed* ili *Compensated*), timeout 30s. Pre-saga odbijanja ne prave log, odgovor je samo HTTP greška.

Assert. Stanje učesničkih redova (računi kupca i prodavca, holdings, ugovor), red u logu, broj redova u *bank.transactions*, i invarijante:

- **I1.** Po valuti: $SUM(available + reserved)$ jednako prethodnom stanju. Transferi unutar iste valute održavaju zbir. (Bez ovoga sistem stvara ili uništava novac.)
- **I2.** Po simbolu: $SUM(quantity + reserved_quantity)$ nepromenjen. (Bez ovoga sistem stvara ili uništava hartije.)
- **I3.** Na kraju toka sve *reserved_** kolone relevantnih redova su 0. (Bez ovoga rezervacije ostaju zarobljene neograničeno.)
- **I4.** Log sadrži zapis za svaki pokušani korak, redom. (Bez ovoga koordinator posle restarta ne ume da nastavi forward niti da završi kompenzaciju.)
- **I5.** Terminalni status je *Completed* ili *Compensated*. (Tokovi zaglavljani u *Running* ili *Compensating* zahtevaju ručnu intervenciju i ne smeju se javiti pod normalnim greškama.)
- **I6.** Ugovor više nije "važeći" samo ako saga završi *Completed*. (Bez ovoga isti ugovor može biti iskorišćen više puta.)

Fault injection headeri

Test koji proverava ponašanje pod greškom šalje adversarne headere na zahtev koji pokreće tok.

X-Saga-Force-Fail: F_i: orkestrator obara imenovanu fazu.

X-Saga-Force-Fail-Kind: before | after: fazu obara pre ili posle primene bočnih efekata.

X-Saga-Compensate-Fail: C_i: orkestrator obara dati kompenzator.

X-Saga-Compensate-Fail-Times: N: broj uzastopnih neuspeha pre nego što kompenzator počne da uspeva.

X-Saga-Inject-Delay: F_i;Nms: pauza od N milisekundi unutar date faze.

Adversarni uslovi i mehanizmi injekcije

- **Adversarni headeri** (*X-Saga-**): hook unutar servisa pravi grešku ili pauzu u fazi koju test imenuje. Dostupan samo u testnom buildu (potencijalno kontrolisano od strane nekog envvar-a); servis odbija da se podigne ako je hook uključen u release modu.

- **Toxiproxy**: TCP proxy ubačen na linkove između servisa (npr. između *trading* i *bank*). Test mu šalje komande za tzv. toksike (*latency, down, bandwidth=0, slicer*) koje simuliraju mrežne probleme bez diranja koda servisa.

- ***docker compose pause/unpause***: pauzira sve procese u container-u preko *SIGSTOP*, nastavlja preko *SIGCONT*. Servis postaje nedostupan ali ne gubi stanje u memoriji niti otvorene konekcije.

- ***docker compose kill -s KILL***: slanje *SIGKILL*-a container-u bez normalnog shutdown-a. Simulira pad procesa. Potrebno je da se pazi na gubitak podataka.

Uslov	Mehanizam	Test
Pozivalac nije kupac iz ugovora	JWT drugog korisnika	SG-02a
Ugovor ne postoji	Poziv sa nepostojećim <i>id</i>	SG-02b
Ugovor nije u statusu "važeći"	Seed: status='iskorišćen' ili 'istekao'	SG-02c
<i>settlementDate</i> prošao	Seed: datum u prošlosti	SG-02d
Kupac nema dovoljno sredstava	Seed: <i>accounts.available</i> < <i>qty</i> × <i>strike</i>	SG-03
Prodavac nema dovoljno hartija	Seed: <i>holdings.quantity</i> < <i>qty</i>	SG-04
Faza pukla posle bočnih efekata	<i>X-Saga-Force-Fail: F_i</i>	SG-05, SG-06, SG-07
Kompenzator pukao N puta, pa uspeo	<i>X-Saga-Compensate-Fail: C_i + -Times: N</i>	SG-08
Servis nedostupan	<i>docker compose pause</i> (servis)	SG-09a, SG-10
Mrežna latencija > RPC timeout-a	Toxiproxy <i>latency</i> Nms (N > orkestratorov <i>timeout</i>)	SG-09b
Mrežna particija	Toxiproxy <i>down</i> ili <i>bandwidth=0</i>	SG-09c
Sporo izvršavanje faze	<i>X-Saga-Inject-Delay: F_i;Nms</i>	SG-10 (setup), SG-11 (setup)
Koordinator ubijen mid-flight	<i>docker compose kill -s KILL trading</i>	SG-11

Format adversarnih headera je na implementaciji.

Primer poziva sa adversarnim headerom

Forsiranje pada F3 sa kompenzacijom C2 koja prvo padne jednom pa uspe:

```
POST /options/C1/exercise HTTP/1.1
```

```
Host: bank.localhost
```

```
Authorization: Bearer <JWT kupca>
```

```
X-Saga-Force-Fail: F3
```

```
X-Saga-Compensate-Fail: C2
```

```
X-Saga-Compensate-Fail-Times: 1
```

```
Content-Type: application/json
```

```
Content-Length: 0
```

Očekivano: F1 i F2 prolaze i upisuju se u log kao "ok", F3 puca, orkestrator pokreće C2 (prvi pokušaj puca jer headerom traženo, drugi pokušaj prolazi), zatim C1 prolazi. Krajnji status je *Compensated*, *current_step* = 3, log ima šest zapisa (F1 ok, F2 ok, F3 err, C2 err, C2 ok, C1 ok). HTTP odgovor sadrži *saga_id* tako da test može da proveriti log.

Testovi

SG-01: "Happy path"

Setup. Ugovor C1 (qty=10, strike=300 USD, settlement=sutra). Kupac ima 5000 USD, prodavac ima 10 jedinica hartije rezervisanih za ugovor.

Action. POST /options/C1/exercise bez headera.

Očekivano. Status *Completed*. Kupac 2000 USD, prodavac 3000 USD, sve hartije kod kupca. Ugovor više nije "važeći". Log: status *Completed*, *current_step*=5, log ima pet "ok" zapisa.

SG-02: Pre-saga validacija (parametrizovano)

Četiri podtesta, jedan po preduslovu. Akcija je ista u svakom, setup razlikuje samo preduslov:

- **a.** Pozivalac nije kupac iz ugovora.
- **b.** *id* ne postoji.
- **c.** Ugovor sa statusom "iskorišćen" ili "istekao".
- **d.** Ugovor sa *settlementDate* u prošlosti.

Očekivano. HTTP greška 4xx. Log za *id* ne postoji. Stanje računa i hartija nepromenjeno.

SG-03: Nedovoljno sredstava (F1 neuspeh)

Setup. Ugovor traži 3000 USD, kupac ima 500 USD.

Action. POST .../exercise.

Očekivano. Žurnal sa statusom *Compensated*, *current_step*=1, log sadrži samo F1 sa greškom. Nema bočnih efekata.

SG-04: Nedovoljno hartija (F2 neuspeh)

Setup. Ugovor traži 10 hartija, prodavac ima 3.

Action. POST .../exercise.

Očekivano. *Compensated*, *current_step*=2. Log: F1 ok, F2 greška, C1 ok. Stanje računa nepromenjeno (rezervacija sa F1 otpuštena u C1).

SG-05: F3 neuspeh, kompenzacija C2 pa C1

Setup. Ugovor spreman.

Action. POST .../exercise + *X-Saga-Force-Fail: F3*.

Očekivano. *Compensated, current_step=3*. Log: F1 ok, F2 ok, F3 greška, C2 ok, C1 ok. Stanje identično prethodnom (invarijante).

SG-06: F4 neuspeh, kompenzacija C3, pa C2, pa C1

Action. Kao Test 5, ali *X-Saga-Force-Fail: F4*.

Očekivano. *Compensated, current_step=4*. Log: F1-F3 ok, F4 greška, C3 ok (sredstva vraćena kupcu), C2 ok, C1 ok. Stanje identično prethodnom.

SG-07: F5 neuspeh, kompenzacija C4, C3, C2, C1

Action. Kao Test 5 ali *X-Saga-Force-Fail: F5*.

Očekivano. *Compensated, current_step=5*. Log: F1-F4 ok, F5 greška, C4 ok (hartije vraćene prodavcu), C3 ok (sredstva vraćena kupcu), C2 ok, C1 ok. Stanje identično prethodnom.

SG-08: Kompenzator pao jednom, pa uspeo

Action. *X-Saga-Force-Fail: F3 + X-Saga-Compensate-Fail: C2 + X-Saga-Compensate-Fail-Times: 1*.

Očekivano. *Compensated*. Log uključuje dva zapisa za C2 (prvi sa greškom, drugi ok). Stanje identično prethodnom.

SG-09: Infrastrukturni neuspeh na ranoj fazi (parametrizovano)

Tri podtesta sa istom akcijom; setup razlikuje injekciju neuspeha pre poziva:

- **a.** docker compose pause bank; bank ostaje pauziran dok se test ne završi.
- **b.** Toxiproxy ispred bank servisa sa *latency* Nms, gde je N veće od orkestratorovog RPC timeout-a.
- **c.** Toxiproxy down ili bandwidth=0 na bank.

Action. POST /options/C1/exercise. Posle završetka cleanup: docker compose unpause bank i/ili uklanjanje Toxiproxy-a.

Očekivano. F1 vraća grešku iz infrastrukture (*Unavailable* za a/c, *DeadlineExceeded* za b). Saga ide u *Compensating* i odmah u *Compensated* jer nema prethodnih faza za kompenzaciju. Log: status *Compensated, current_step=1*, sadrži samo F1 sa greškom. Stanje računa i hartija nepromenjeno.

SG-10: Servis pauziran tokom sage

Setup. Ugovor C1 spreman. Header *X-Saga-Inject-Delay: F3:5000* da otvori prozor između F2 i F3.

Action. POST .../C1/exercise. Tokom prozora test izvršava docker compose pause bank. Posle F3 timeout-a saga ulazi u *Compensating*; test odpauzira bank pre nego što kompenzator C1 bude pozvan (npr. čim zapazi *current_step=3* i status=*Compensating* u logu).

Očekivano. F3 vraća *Unavailable*. C2 prolazi (trading je neometan), C1 prolazi (bank je u međuvremenu odpauziran). Tok završi u *Compensated, current_step=3*, log: F1 ok, F2 ok, F3 greška, C2 ok, C1 ok. Stanje identično prethodnom.

SG-11: Pad koordinatora mid-flight

Setup. Ugovor spreman. Header *X-Saga-Inject-Delay: F3:5000*.

Action. POST .../C1/exercise. Test čeka 1s, izvršava docker compose kill -s KILL trading, pa docker compose up -d trading.

Očekivano. Posle restarta koordinator pročitava log i nastavi tok. Krajnji status je *Completed* ili *Compensated* (deterministički zavisi od tačke kada je SIGKILL stigao; oba ishoda su validna ako su invarijante zadovoljene). Log je konzistentan, nema visećih rezervacija.